Multimedia Signal Processing - 2nd Module

September, 12 2016

1. **Question (6 pts)**
   Consider the following sequence of samples:

$$x = \{10,\ 12,\ 15,\ 16,\ 20,\ 19,\ 18,\ 15,\ 13\}.$$

   (a) Obtain the sequence $\tilde{x}$ encoding x using PCM with the following quantizer:

$$\begin{array}{ll} I_1 = [8, 12), & y_1 = 10 \\ I_2 = [12, 16), & y_2 = 14 \\ I_3 = [16, 20), & y_3 = 18 \\ I_4 = [20, 24), & y_4 = 22 \end{array}$$

   (b) Compute the mean squared error (MSE) introduced by the use of PCM.

   (c) Estimate the pmf of each symbol of $\tilde{x}$ and derive the Huffman coding scheme.

   (d) Obtain the sequence $\hat{x}$ encoding x using a closed-loop DPCM with the following quantizer:

$$\begin{array}{ll} I_1 = [-\infty, -1.5), & y_1 = -3 \\ I_2 = [-1.5, 1.5), & y_2 = 0 \\ I_3 = [1.5, +\infty), & y_3 = 3 \end{array}$$

   Consider that the first symbol is already known by the decoder.

   (e) Compute the mean squared error (MSE) introduced by the use of DPCM.

Solution

   (a) According to the provided quantization rules, the sequence becomes

$$\tilde{x} = \{10,\ 14,\ 14,\ 18,\ 22,\ 18,\ 18,\ 14,\ 14\}.$$

   (b) Let us compute the sample-by-sample absolute error

$$e = |x - \tilde{x}| = \{0,\ 2,\ 1,\ 2,\ 2,\ 1,\ 0,\ 1,\ 1\}.$$

   The MSE is

$$\text{MSE} = \frac{1}{9}\sum_{i=1}^{9} e_i^2 = \frac{16}{9}.$$

(c) The pmf is estimated by counting and normalizing symbols occurrences. The alphabet of symbols is $\{10, 14, 18, 22\}$. The resulting pmf is $\tilde{x}$ can be computed as follows

$$p_{10} = 1/9, \quad p_{14} = 4/9; \quad p_{18} = 3/9; \quad p_{22} = 1/9.$$

A possible Huffman code becomes

$$C_{10} = 000, \quad C_{14} = 1; \quad C_{18} = 01; \quad C_{22} = 001.$$

Notice that symbols with higher probability are described by shorter codes.

(d) Let us denote with $r_i = x_i - \hat{x}_{i-1}$ the residual, and with $\hat{r}$ its quantized version. The decoded sequence $\hat{x}$ can be computed as follows

x	10	12	15	16	20	19	18	15	13
r	-	2	2	0	4	0	-1	-4	-3
$\hat{r}$	-	3	3	0	3	0	0	-3	-3
$\hat{x}$	10	13	16	16	19	19	19	16	13

(e) Let us compute the sample-by-sample absolute error

$$e = |x - \hat{x}| = \{0,\ 1,\ 1,\ 0,\ 1,\ 0,\ 1,\ 1,\ 0\}.$$

The MSE is

$$\text{MSE} = \frac{1}{9}\sum_{i=1}^{9} e_i^2 = \frac{5}{9}.$$

2. **Question (6 pts)**

(a) Introduce the concept of scalar quantizer, showing the difference between a midtread and a midrise one.

(b) Describe the compander/expander operation, highlighting how quantization can benefit from it.

(c) Introduce the concept of vector quantization.

Solution See lecture slides.

3. **Question (6 pts)**

(a) Introduce the concept of perceptual audio coding.

(b) Describe the MPEG1 psychoacoustic model 1 for global masking threshold computation.

Solution See lecture slides.

4. **Questions (5 pts - each answer can be either TRUE or FALSE)**
   (a) Consider a video coding scheme.
   - [T] [F] The displaced frame difference is computed for all the frames of a sequence.
     FALSE. At least one is intra-coded.
   - [T] [F] Motion compensation is the process of predicting motion vectors.
     FALSE. It's motion estimation. Compensation applies motion vectors to compute the DFD.
   - [T] [F] It is possible to estimate motion vectors with sub-pixel precision.
     TRUE.
   - [T] [F] DCT is used for both inter- and intra-coded frames.
     TRUE.

   (b) Consider the JPEG coding standard.
   - [T] [F] DCT is applied separately on red, green and blue channels.
     FALSE. JPEG works in YCbCr.
   - [T] [F] Zig-zag scan is used to apply run-length coding to quantized DCT coefficients.
     TRUE. Only for AC components.
   - [T] [F] 4x4 DCT is used for sub-sampled color components.
     FALSE. It's 8x8.

   (c) Consider two discrete memoryless sources X and Y.
   - [T] [F] Mutual information $I(X,Y)$ can be negative only for independent sources.
     FALSE. It cannot be negative.
   - [T] [F] Conditional entropy $H(X|Y)$ is always lower than $H(X)$.
     FALSE. It can be equal.
   - [T] [F] Joint entropy $H(X;Y)$ can be lower than $H(X)$.
     FALSE.

5. **Question (Matlab - 10pts)**
   (a) Load the image `lena512color.tiff` and display it in RGB. From now on, consider the image as only composed by the first two color channels (`HxWx2`).

   (b) Compute the entropies of the first two color channels separately (consider each channel as a monodimensional signal).

   (c) Compute the joint entropy of the first two channels (consider the image as a list of pixels, each one composed by two elements only).

   (d) Implement the following hybrid DPCM - transform coding scheme:
   - The first channel is transform coded using 16x16 DCT and quantizer with $\Delta = 5$.
   - The decoded first channel is subtracted to the second channel to obtain the second channel's residual.
   - This residual is transform coded using 16x16 DCT and quantizer with $\Delta = 5$.
   - Finally the second channel is decoded summing the decoded residual to the decoded first channel.

(e) Compute for each channel the PSNR achieved after decoding and the coding rate (in terms of entropy).

Solution

```
clc, clear all, close all


%% Load and display image
I = imread('lena512color.tiff');

figure(1)
imshow(I), title('Image')


%% Compute the entropy of the first two color channels
H_c = [0 0];

%%% For each channel
for c = 1:2
    %%% Select channel
    I_c = I(:,:,c);
    I_c = I_c(:);

    %%% Compute pmf
    hist_c = hist(I_c, 0:255);
    hist_c(hist_c==0) = [];
    p_c = hist_c/sum(hist_c);

    %%% Compute entropy
    H_c(c) = -sum(p_c.*log2(p_c));
    fprintf('The entropy of channel %d is: %.3f [bit/pixel]\n', c, H_c(c));
end


%% Compute the entropy of each pixel (considering only two color channels)
[H, W] = size(I(:,:,1));
N_pixels = H*W;

%%% Reshape pixels to have a list of symbols
R = I(:,:,1);
G = I(:,:,2);
symbols = [R(:) G(:)];

%%% Compute histogram / pmf
hist_I = hist3(symbols, {0:255 0:255});
hist_I = hist_I(:);
hist_I(hist_I == 0) = [];
p_I = hist_I/sum(hist_I);

%%% Compute entropy
H_I = -sum(p_I.*log2(p_I));
fprintf('The entropy of joint channels 1 and 2 is: %.3f [bit/pixel]\n', H_I);
fprintf('The sum of each channel''s entropy is: %.3f [bit/pixel]\n', sum(H_c));


%% Implement the described coding scheme

%%% Parameters
B = 16; %block size
delta = 5; %quantization step

%%% Encode and decode I_1
I_1 = double(I(:,:,1));
I_1_dec = zeros(size(I_1));
I_1_enc = zeros(size(I_1));

[H, W] = size(I_1);
for h = 1:B:H-B+1
    for w = 1:B:W-B+1
        block = I_1(h:h+B-1, w:w+B-1);
        block_dct = dct2(block);
        block_dct_q = delta.*round(block_dct/delta);
        block_idct = idct2(block_dct_q);
        I_1_enc(h:h+B-1, w:w+B-1) = block_dct_q;
        I_1_dec(h:h+B-1, w:w+B-1) = block_idct;
    end
end

%%% Some Plot
figure(2)
subplot(231), imagesc(I_1, [0 255]), axis image, title('I_1')
subplot(232), imagesc(I_1_dec, [0 255]), axis image, title('I_{1,dec}')
subplot(233), imagesc(abs(I_1 - I_1_dec), [-10 10]), axis image, title('I_1 - I_{1,dec}')

%%% Compute PSNR
MSE_1 = mean( (I_1(:) - I_1_dec(:)).^2 );
PSNR_1 = 20*log10(255/sqrt(MSE_1));
```

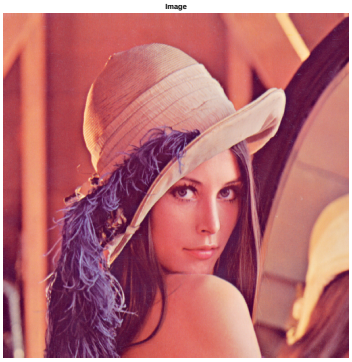
```
%%% Compute Rate (in the coded domain)
bins_1 = unique(I_1_enc(:));
hist_1 = hist(I_1_enc,bins_1);
p_1 = hist_1/(sum(hist_1));
H_1 = -sum(p_1.*log2(p_1));
fprintf('First frame: PSNR=%.2f, Rate=%.2f\n', PSNR_1, H_1);

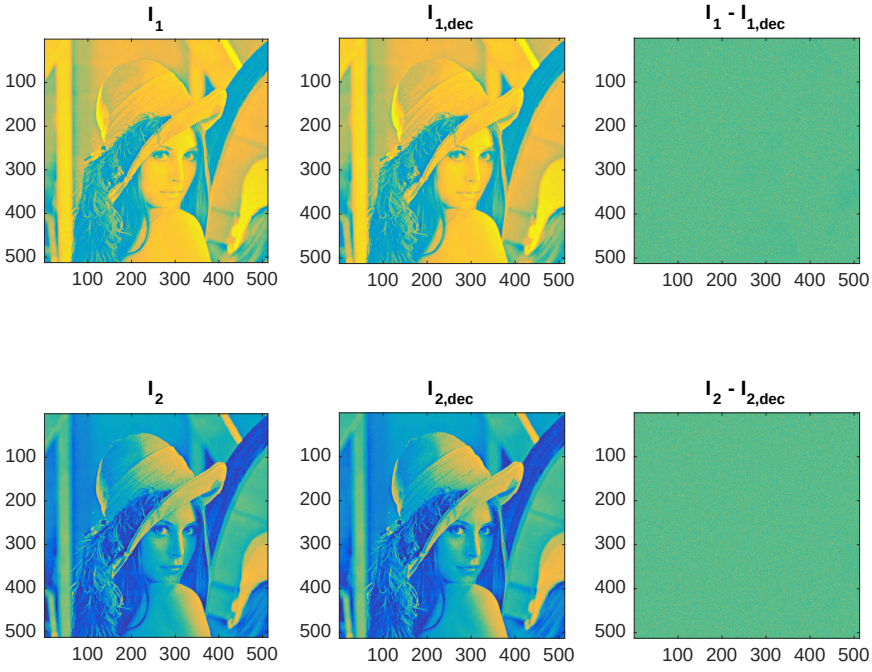
%%% Encode and decode I_2
I_2 = double(I(:,:,2));
I_2_res = I_2 - I_1_dec;
I_2_res_enc = zeros(size(I_2));
I_2_res_dec = zeros(size(I_2));
[H, W] = size(I_2);
for h = 1:B:H-B+1
    for w = 1:B:W-B+1
        block = I_2_res(h:h+B-1, w:w+B-1);
        block_dct = dct2(block);
        block_dct_q = delta.*round(block_dct/delta);
        block_idct = idct2(block_dct_q);
        I_2_res_enc(h:h+B-1, w:w+B-1) = block_dct_q;
        I_2_res_dec(h:h+B-1, w:w+B-1) = block_idct;
    end
end
I_2_dec = I_2_res_dec + I_1_dec;

%%% Some plot
figure(2)
subplot(234), imagesc(I_2, [0 255]), axis image, title('I_2')
subplot(235), imagesc(I_2_dec, [0 255]), axis image, title('I_{2,dec}')
subplot(236), imagesc(abs(I_2 - I_2_dec), [-10 10]), axis image, title('I_2 - I_{2,dec}')

%%% Compute PSNR (in the coded domain)
MSE_2 = mean( (I_2(:) - I_2_dec(:)).^2 );
PSNR_2 = 20*log10(255/sqrt(MSE_2));

%%% Compute rate
bins_2 = unique(I_2_res_enc(:));
hist_2 = hist(I_2_res_enc,bins_2);
p_2 = hist_2/(sum(hist_2));
H_2 = -sum(p_2.*log2(p_2));
fprintf('Second frame: PSNR=%.2f, Rate=%.2f\n', PSNR_2, H_2);
```


Image


I 1
I 1,dec
I 1 - I 1,dec
I 2
I 2,dec
I 2 - I 2,dec
100
200
300
400
500